

DOCUMENTO TÉCNICO: PROYECTO CIVILIZATIONS

M01 - Sistemas Informáticos / Proyecto Integrado



**AMS 1
CINTHIA VALERIA
DIEGO CRUZADO
MIGUEL ORTIZ**

ÍNDICE GENERAL

1. Introducción del proyecto	Pág. 1
2. Análisis y diseño	Pág. 1
<i>Especificaciones funcionales</i>	
<i>Especificaciones no funcionales</i>	
<i>Estructura de clases principales</i>	
3. Arquitectura y desarrollo	Pág. 2
<i>Estructura del código fuente (Back-End y Front-End)</i>	
<i>Flujo de datos y eventos concurrentes</i>	
4. Test y depuración	Pág. 3
<i>Pruebas realizadas</i>	
<i>Errores encontrados y soluciones aplicadas</i>	
5. Recursos gráficos y de audio	Pág. 3
6. Manual de usuario	Pág. 4
<i>Instrucciones de ejecución</i>	
<i>Controles y mecánica de juego</i>	
7. Reflexión y mejoras	Pág. 4
<i>Dificultades con el control de versiones en GitHub</i>	
<i>Funcionalidades y mejoras futuras</i>	
8. Conclusiones finales	Pág. 5

1. Introducción del proyecto

Descripción breve: Aplicación interactiva de estrategia que simula la gestión, desarrollo económico y defensa militar de una civilización bajo constantes asedios externos .

Objetivos principales: Desarrollar la lógica de negocio orientada a objetos en Java, implementar automatización por eventos mediante hilos concurrentes, garantizar la

persistencia de datos en entornos distribuidos y desplegar una plataforma web informativa conectada .

Eines i entorns utilitzats:

IDE y Desarrollo: Eclipse IDE para el diseño visual de pantallas y VS Code para la codificación ágil de scripts y maquetación web .

Lenguaje y Frameworks: Java SE (JDK) y la librería gráfica nativa Swing.

Arquitectura de Datos: Motor de base de datos Oracle alojado en un nodo de virtualización Proxmox .

Frontend Web: Entorno Node.js utilizando el motor de plantillas dinámicas .hbs.

Control de versiones: Herramienta Git y repositorio remoto en la plataforma GitHub.

2. Anàlisi i disseny

Especificaciones funcionales

Gestión de economía: Generación periódica automática de Comida, Madera y Hierro . El Maná se genera exclusivamente de forma condicionada si se dispone de al menos una Torre Mágica activa.

Infraestructura: Compra e instalación de Granjas, Carpinterías, Herrerías, Iglesias y Torres Mágicas con validación rígida de costes .

Reclutamiento de unidades: Compra de flotas ofensivas (Espadachín, Lancero, Ballesta, Cañón) , defensas estáticas (Torre lanza, Catapulta, Torre lanzacohetes) y unidades especiales de soporte (Mago, Sacerdote) .

Simulador de batalla: Enfrentamientos automáticos regidos por porcentajes de probabilidad . Los combates finalizan cuando cualquier bando se reduce al 20% o menos de sus efectivos iniciales . El vencedor se determina mediante el cálculo exacto de pérdidas materiales ponderadas .

Especificaciones no funcionales

Robustez: Control estricto de errores e imprevistos lógicos mediante herencia de excepciones personalizadas (ResourceException y BuildingException).

Estructuración: Modularidad de código dividida de manera estricta en carpetas independientes para cada módulo académico evaluado (M1, M2, M3, M4, M5).

Equilibrio de juego: Centralización y parametrización absoluta de la dificultad y modificadores numéricos en la interfaz estática Variables.

Estructura de clases principales

Civilizations: Clase contenedora del estado del jugador. Gestiona contadores de edificios, almacenes de recursos, tecnologías y el array estructurado de listas `ArrayList<MilitaryUnit>[] army`.

Battle: Motor algorítmico del combate por turnos . Controla las fases de ataque, selección aleatoria ponderada de defensores, cálculo de escombros y almacenamiento del log detallado (battleDevelopement .

Interfaz MilitaryUnit: Define el contrato común para todas las unidades del software (métodos attack(), takeDamage(), costes de recursos y restauraciones) .

3. Arquitectura i desenvolupament

Estructura del código fuente (Back-End y Front-End)

El código sigue una arquitectura multicapa distribuida en paquetes lógicos que aíslan la interfaz de usuario de la base de datos:

exceptions: Contiene ResourceException.java y BuildingException.java .

interface/: Define MilitaryUnit.java y la interfaz de constantes paramétricas Variables.java.

models/: Alberga las abstracciones intermedias (AttackUnit, DefenseUnit, SpecialUnit) (Proyecte junto con las implementaciones específicas de cada unidad (p. ej., Sworsman.java, Catapult.java).

gui/: Clases de ventanas desarrolladas con Swing (MainGUI.java, BattlePanel.java) encargadas del control visual.

web/: Servidor Node.js, controladores de base de datos y plantillas .hbs independientes para el despliegue del portal informativo .

Flujo de datos y eventos concurrentes

Temporizadores en segundo plano: Se utiliza la clase nativa TimerTask para planificar dos tareas asíncronas concurrentes controladas por un único objeto Timertask .

Hilos económicos: Cada 60 segundos se desencadena la actualización de almacenes sumando los recursos base incrementados por los multiplicadores de los edificios construidos (un 10% adicional por cada granja, carpintería o herrería) .

Hilos de invasión: Cada 3 minutos se instancia automáticamente el método createenemyArmy(), generando una nueva amenaza con recursos que escalan de manera geométrica según el histórico de batallas jugadas .

Flujo Post-Batalla: Al concluir un combate, el método `updateResourceLoose()` evalúa las pérdidas). Si la civilización vence, se realiza un volcado de datos que incrementa los almacenes locales con los residuos metálicos recolectados en el terreno .

4. Test i depuració

Pruebas realizadas

Se ejecutaron bancos de pruebas manuales y unitarias enfocados en el balanceo y la estabilidad lógica de los combates:

Validación de excepciones: Verificación empírica de que al solicitar la creación de 10 unidades teniendo recursos únicamente para 5, el sistema añade exitosamente las 5 posibles, congela la operación y lanza la excepción controlada al usuario .

Prueba de hilos concurrentes: Monitorización de la UI para garantizar que las ventanas de Swing no sufrieran congelamientos visuales mientras los hilos `TimeTask` `ResourceException` actualizaban los datos en segundo plano .

Errores encontrados y soluciones aplicadas

Errores de compilación en tipos complejos: Durante las fases iniciales se presentaron fallos de tipos al intentar instanciar el array de colecciones genéricas `ArrayList<MilitaryUnit>[] army = new ArrayList[9]`. Se solucionó suprimiendo las advertencias de conversión mediante anotaciones y forzando el casting explícito seguro en los métodos de reclutamiento.

Fallas de conectividad e hilos bloqueantes: Al realizar transacciones CRUD directas hacia la base de datos Oracle en Proxmox utilizando Swing en el hilo principal (*Event Dispatch Thread*), la interfaz gráfica se bloqueaba por completo si existía latencia de red . La incidencia se mitigó abstrayendo las consultas de actualización dentro de tareas en segundo plano mediante la clase `MainGUI`

Pérdida de sincronización de datos en tiempo real: Los valores reflejados en la web no coincidían con el juego activo. Se solucionó reestructurando la lógica para forzar un guardado automático (`commit SQL`) inmediato en la base de datos tras la finalización exacta de cada combate o evento de construcción .

5. Recursos gràfics i d'àudio

Integración en la interfaz de usuario Swing

Diseño Visual: Implementado mediante ventanas JFrame, paneles JPanel y componentes interactivos JButton y JLabel. Se utilizaron administradores de diseño (*Layout Managers*) como BorderLayout y GridWayLayout para asegurar una correcta distribución espacial de los marcadores de recursos.

Recursos Gráficos: Incorporación de iconos personalizados en formato .png para representar la comida, madera, hierro y maná , integrados mediante la clase Imagon sobre componentes Swing. Estos recursos mantienen una coherencia estética idéntica a las imágenes dinámicas servidas en las plantillas .hbs de la plataforma web .

Gestión de rutas: Para evitar errores de carga al ejecutar el juego en diferentes sistemas operativos (Linux/Windows), todos los elementos visuales se empaquetaron dentro del directorio de recursos (src/resources/) y se cargaron en memoria utilizando flujos de lectura relativos de tipo getResourcesAsExtrem().

6. Manual d'usuari

Instrucciones de ejecución

Para lanzar la aplicación gráfica del videojuego desde la terminal de comandos de Linux, sitúate en la raíz del proyecto empaquetado y ejecuta:

bash

```
java -jar CivilizationsGame.jar
```

Controles y mecánica de juego

Pantalla Principal: Monitoriza de manera constante tus almacenes de recursos (Comida, Madera, Hierro, Maná) y tus niveles tecnológicos actuales .

Panel de Construcción: Permite expandir tu número de Granjas, Carpinterías y Herrerías para optimizar la recolección automática por minuto . Nota: Debes construir un mínimo de una Torre Mágica si deseas empezar a acumular Maná y desbloquear el reclutamiento de la unidad Mago .

Gestión de Cuartel: Botones interactivos para reclutar tropas ofensivas y defensivas ingresando la cantidad deseada

Gestión de Invasiones: El juego te notificará visualmente cada 3 minutos ante una amenaza inminente new threat commit detallando los efectivos que te atacarán. La batalla se resolverá de forma automatizada e interna, permitiéndote abrir un cuadro de diálogo para inspeccionar el informe paso a paso o el reporte resumido de bajas y escombros recuperados .

7. Reflexió i millores

Dificultades con el control de versiones en GitHub

Conflictos en Merges: Al trabajar en ramas de desarrollo individuales e intentar consolidar los cambios en la rama intermedia de preproducción, se generaron múltiples conflictos de fusión (*merge conflicts*) en archivos compartidos críticamente como Main.java o Vaeriable.java.

Resolución: Se implementó una política estricta de comunicación en el equipo que obligaba a realizar operaciones de git pull preproducción antes de iniciar cualquier jornada de desarrollo, limpiando y solventando los conflictos de manera manual directamente desde los editores VS Code o Eclipse antes de confirmar cualquier envío git commit

Funcionalidades y mejoras futuras

Coste de mantenimiento económico: Implementar un sistema de consumo continuo donde mantener tropas avanzadas (como el Espadachín o Lancero) requiera un flujo constante de Comida deducible por minuto, provocando la desertión o muerte de las unidades si los almacenes bajan a cero .

Limitación tecnológica: Restringir mediante un tope máximo las actualizaciones de las tecnologías de ataque y defensa para evitar desbalances insalvables en fases avanzadas de la partida .

Sistema de restauración pagado: Modificar la lógica para que la acción de restablecer las armaduras y escudos dañados de las flotas supervivientes tras una batalla requiera una inversión activa de Hierro y Madera, añadiendo una capa estratégica adicional al juego

8. Conclusiones finales

La realización del proyecto integral *Civilizations* ha permitido consolidar de manera práctica las competencias transversales del desarrollo de software multi-módulo. A través de la documentación y la resolución del software, se extraen las siguientes conclusiones clave:

Integración Multidisciplinar Exitosa: El proyecto demuestra que el desarrollo de un videojuego interactivo no se limita a la programación de su lógica aislada . Exige una simbiosis perfecta con la administración de sistemas (M01) para la gestión de recursos de ejecución en Linux, el modelado de datos persistentes (M02) mediante bases de datos relacionales, y el despliegue de plataformas de marcas (M04) que expongan de manera atractiva el estado del juego al exterior .

Importancia del Control de Concurrencia: La implementación de hilos independientes a través de TimerTask evidenció los desafíos de los entornos multitarea . Garantizar la sincronización exacta entre los contadores económicos de fondo, las rutinas automáticas de ataque enemigo y el refresco visual de las ventanas diseñadas en Swing supuso un aprendizaje fundamental para evitar hilos bloqueantes e inconsistencias de datos en memoria.

Madurez en el Trabajo Colaborativo: Las dificultades experimentadas durante las fusiones de ramas en GitHub reafirmaron que la comunicación humana y el establecimiento previo de políticas de desarrollo estrictas son tan críticos como la calidad del código escrito . El uso coordinado de herramientas en entornos como Eclipse y VS Code permitió estandarizar los flujos de integración y resolver conflictos de forma metódica.